

for..of can also be used to iterate through the arguments object of a function/method. Note that *arguments* is a local variable available within all functions.

Snippet 3: Example to demonstrate arguments object

```
1. function fn(one, two, three) {
2. for(let arg of arguments) {
   console.log(arg);
3. }
4. }
5. fn("mon", "tue", "wed");
Output:
mon
tue
wed
```

```
let makept = (a, b) => ({ x:a, y:b});
```

Arrow function that returns an object.

```
let capcat = (a, b) => {
  let c = a.toUpperCase()
  c += " " + b.toUpperCase();
  return c;
}
```

Arrow function with body and an explicit return at the end.

In addition to strings, arrays, arguments and new collections, *for..of* can also be useful with DOM manipulation.

To provide this capability of iterating through values, built-in functions have been added to the array object, i.e., the prototype method of string and array objects has the *Symbol.iterator* method. This new *Symbol.iterator* introduction, in combination with other new features such as generators, makes this new *for..of* statement powerful.

Collections

Maps and Sets are two new categories of collections added to ES6. Maps allow the storing of arbitrary values as a collection, and Sets is used for storing a collection of unique elements.

Map is composed of key-value pairs. Language supports built-in operations of *set()*, *get()*, *has()* and *delete()*.

Set has operations of *add()*, *clear()* and *has()* to add elements, delete all elements and check the presence of an element, respectively.

Two other variants of Map and Set are the WeakMap and WeakSet. The Weak variants of collections have a few restrictions. Weak variants cannot be iterable, which means we cannot use the *for..of* statement. The second

limitation is that we can only store objects and not primitive data types.

One reason for introducing WeakMap and WeakSet is automatic garbage collection. In the case of Map and Set, the programmer needs to take responsibility for checking references and clearing the memory.

Arrow functions

A new shorthand notation for anonymous functions is the arrow function. It is represented by the symbol $\Rightarrow$. This notation of equal-to instead of a dash (or hyphen) makes it a fat arrow function. This notation has been borrowed from CoffeeScript, and is extremely useful when defining callback functions. Arrow functions have more than cosmetic features. They overcome some of the side effects of this binding with traditional functions. In addition, they also help JS engines to perform better with something called 'tail call optimisations'. For details, go to *References* at the end of the article.

Let's look at an example of an anonymous function definition like the one below:

```
1. var std = function std(yr) {
2.   return "ECMAScript" + yr;
3. };
4. console.log(std("2015"));
```

With the ES6 arrow function, the same can be shortened to:

```
1. var std = yr => "ECMAScript"+yr;
2. console.log(std("2015"));
```

Output:

```
ECMAScript2015
```

Points to note in the new syntax are that the keyword function has been replaced by the $\Rightarrow$ symbol, and there are no parentheses around the parameter. Since the method has only one statement (expression), there are no curly brackets to enclose the function body. There is no return statement as the body of the function is an expression.

Given below are a few examples of arrow functions, with varying number of parameters and body of the function.

```
let x = () => {};
```

Arrow function with no parameters and no body.

```
let y = (a, b) => a + b;
```